Advanced Git Operations

Module 3 — Onboarding & Industrialize the Codebase

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

Until now, we only used the git basics commands. Today we add four operations of everyday team work, and apply them to `heron`:

- the team's **contribution rules**, in a `CONTRIBUTING.md` file
- **merge conflicts**: why they happen and how to resolve them
- **`git bisect`**: find the commit that broke the code
- **tags**: give a release a name

Some commands beyond clone, commit, push, pull

These commands come up in almost every lecture:

```
git switch -c docs/quality-checks   # create a branch and move to it (clearer than `git checkout -b`)
git switch lab-01-start             # move to an existing branch
git restore README.md               # throw away the uncommitted changes to one file
git diff lab-01-start -- README.md  # what changed in a file since a branch or commit
git stash                           # put uncommitted work aside...
git stash pop                       # ...and bring it back
git log --oneline --graph --all     # the history as a readable tree
```

Undo a mistake (1/2)

Each common mistake has one safe command:

The mistake	The command
I changed a file and I want the committed version back	`git restore <file>`
I added a file to the next commit by mistake	`git restore --staged <file>`
My last commit (not pushed) misses a file or has a wrong message	`git add <file>`, then `git commit --amend`
I want to undo my last commit (not pushed) but keep its changes	`git reset --soft HEAD~1`

Never `--amend` or `reset` a commit that you have already pushed.

Undo a mistake (2/2)

Each common mistake has one safe command:

The mistake	The command
A commit that I already pushed is wrong	`git revert <commit-id>`
I lost a commit (after a reset, or a switch to the wrong branch)	`git reflog`, then `git switch -c rescue <commit-id>`

`git revert` does not delete the wrong commit: it adds a new commit that undoes it. Use it for pushed commits, because other clones already have them. `git reflog` lists every commit your `HEAD` has pointed to in the last weeks, so a commit you "lost" is almost always still there.

Contribution rules

CONTRIBUTING.md

The README tells a **user** how to install and run the project. A `CONTRIBUTING.md` file, at the root of the repository, tells a **contributor** how the team works: how to name a branch, how a change gets merged, how to write a commit message. GitHub links it automatically when someone opens an issue or a pull request.

Each rule should be short and checkable: "write good code" is not a rule; "one branch per change, named `<type>/<short-name>`" is.

Step 1: the contribution rules

Create `CONTRIBUTING.md` at the root of the repository, in your own words, with these four rules:

```
# Contributing

- One branch per change, named `<type>/<short-name>`, for example
  `feat/json-output` or `docs/quality-checks`.
- Every change reaches the base branch through a pull request, and is merged
  only when the checks pass.
- Use [Conventional Commits](https://www.conventionalcommits.org):
  `feat: ...`, `fix: ...`, `docs: ...`, `test: ...`, `refactor: ...`, `chore: ...`.
- Keep commits small and focused; write the *why* in the commit body.
```

```
git add CONTRIBUTING.md
git commit -m "docs: add contribution guidelines"   # the pre-commit hooks run
git push
```

Merge conflicts

When git cannot merge

A **pull request** (PR) is a request to merge one branch into another, with a page on GitHub where the team reads the diff and comments on it before the change lands.

A merge combines two lines of history. Git does it by itself when the two sides changed **different** lines, even in the same file. When both sides changed the **same lines** in different ways, git cannot choose: it stops with a **conflict**, and a person decides.

A conflict happens in every merge-like command: `git merge`, `git pull` (which is `git fetch` followed by a merge), `git rebase`, `git stash pop`.

The most common case in a team: you and a colleague change the same lines on the same branch. One of you pushes first; the other must **pull** before pushing, and the pull merges the two changes.

Demo: one line, changed in two places

I change the **first line** of `README.md` (the title) on `lab-01-start` twice, in two different ways:

1. on GitHub, in my fork: the pencil icon on `README.md`, a new title, *Commit changes*
2. in PyCharm, in my clone, without pulling first: another title, then commit

```
git push                    # rejected: the fork has a commit that my clone does not have
git pull --no-rebase        # fetch + merge: CONFLICT (content): Merge conflict in README.md
git status                  # "both modified: README.md"
```

`--no-rebase` asks for a merge. Recent versions of git refuse a `git pull` on diverged branches until you choose merge or rebase, once per command or once for all with `git config pull.rebase false`.

Resolve the conflict

Git writes **both** versions into the file, between markers:

```
<<<<<<< HEAD
# heron: triage suspicious e-mails
=======
# heron, a phishing triage tool
>>>>>>> 3f2c1a7...
```

`HEAD` is your version, the part after `=======` is the one you pulled. Edit the file: keep one version, or combine them, and **delete the three marker lines**. Then finish the merge:

```
git add README.md
git commit                    # git proposes the message "Merge branch ..."
git push
```

PyCharm shows the same conflict in three panels (*Git → Resolve Conflicts*): your version on the left, the pulled one on the right, the result in the middle. Never commit a file that still contains `<<<<<<<`: the pre-commit hooks do not check it, but every reader will see it.

Fewer conflicts

Conflicts are normal in a team, but small ones are easy and big ones are not:

- pull often, and push small commits often: a conflict on one line is quick to resolve; a conflict on a week of work is not
- one branch per change (the rule of `CONTRIBUTING.md`): two people rarely touch the same lines. Remember the Git workflow: one issue and linked branch+PRs for each feature/fix.
- do not mix a reformatting with a change of behaviour: a `style:` commit that touches every line conflicts with everything
- when the conflict is in code, run the checks and the program **after** resolving it: a merge can be wrong even when it has no markers

Finding a regression

git bisect: binary search in the history

A **regression** is a behaviour that worked before and is now broken. When you do not know which commit broke it, `git bisect` finds it by **binary search**:

1. you name one **bad** commit (the behaviour is broken) and one **good** commit (it works)
2. git checks out the commit in the middle; you test it and answer `git bisect good` or `git bisect bad`
3. git halves the interval and repeats, until one commit is left: the first bad one

With n commits between good and bad, you test about $\log_2 n$ of them: 5 tests for 20 commits, 10 for 1000.

```
git bisect start
git bisect bad                 # the current commit is broken
git bisect good <old-id>       # this old commit worked
# ... test, then: git bisect good   or   git bisect bad   (git moves to the next commit)
git bisect reset               # back to where you started
```

Let a script answer: git bisect run

When a command can tell good from bad, `git bisect run <command>` answers for you. Its **exit code** is the answer:

- `0`: good
- `1` to `127` (except `125`): bad
- `125`: "cannot test this commit" (for example, it does not build): git skips it

```
git bisect start HEAD <old-id>        # bad, then good, in one command
git bisect run ./bisect-check.sh      # git runs the script on each commit it checks out
git bisect reset
```

The script can be a test, a run of the program on a sample, or any command whose exit code means "works". A test suite makes the best script: `git bisect run uv run pytest -q`.

Demo: a silent regression in heron

`unica-isde/heron-bisect` is a separate repository with 20 commits of `heron`, the code of the end of *Modularity and Python Modules*. Somewhere in the history, one commit silently changes the score of one sample: `password-reset-urgent.eml` has score 9 at the first commit and less at the last one. Nothing crashes, the verdict stays *phishing*, and there are no tests.

```
git clone https://github.com/unica-isde/heron-bisect.git && cd heron-bisect
git bisect start HEAD $(git rev-list --max-parents=0 HEAD)   # bad = HEAD, good = the first commit
git bisect run ./bisect-check.sh                             # about 5 steps
git show <the-first-bad-id>                                  # read the culprit
git bisect reset
```

`bisect-check.sh` runs the engine on the sample and exits 0 if the score is 9. It needs a `python3` of version 3.11 or newer; with an older one, run `git bisect run uv run --no-project --python 3.12 ./bisect-check.sh`. Try it yourself after the lecture: why does the culprit change the score? *pytest in Practice* comes back to this regression.

Naming a release

Tags

A **tag** is a fixed name for one commit. A branch moves forward with each commit; a tag never moves. Projects tag each **release** with its semver number and a `v` prefix (`v0.3.0`), so "the code of version 0.3.0" is always one command away (`git switch --detach v0.3.0`).

- an **annotated** tag (`git tag -a`) is an object of its own, with an author, a date and a message: use it for releases
- a **lightweight** tag (`git tag name`) is only a name for a commit: fine for a private bookmark
- on GitHub, a **release** is a page built on a tag, with notes and files to download

Step 2: tag version 0.3.0

We have set the heron version to `0.3.0`. Let's tag that commit:

```
git log --oneline --grep "release 0.3.0"                                    # find its ID (replace with commit message)
git tag -a v0.3.0 <release-id> -m "heron 0.3.0: packaged, with quality gates"   # an annotated tag
git tag --list                                                              # all tags
git show v0.3.0                                                             # the tag message, then the commit
```

A plain `git push` does **not** send tags. Push one tag by name, or use `--follow-tags`, which sends, together with the branch, the annotated tags that point to its commits:

```
git push origin v0.3.0
```

A tag is a promise to users: never move or delete a tag that you have pushed. If the release was wrong, publish a new version (`v0.3.1`).

Takeaways

- `CONTRIBUTING.md` writes down how the team works: branch names, pull requests, commit messages
- a conflict means both sides changed the same lines: edit the file, remove the markers, `git add`, `git commit`, and check that the code still works; pull and push small changes often
- `git bisect` finds the first bad commit by binary search, in $\log_2 n$ tests; `git bisect run` lets a script answer, by its exit code
- an annotated tag (`v0.3.0`) names a release; push it explicitly, and never move it

Appendix: every step in full

Step 1 in full

Commit `docs: add contribution guidelines`: one new file. `CONTRIBUTING.md` is written from scratch; yours says the same in your own words.

```
# Contributing

- One branch per change, named `<type>/<short-name>`, for example
  `feat/json-output` or `docs/quality-checks`.
- Every change reaches the base branch through a pull request, and is merged
  only when the checks pass.
- Use [Conventional Commits](https://www.conventionalcommits.org):
  `feat: ...`, `fix: ...`, `docs: ...`, `test: ...`, `refactor: ...`, `chore: ...`.
- Keep commits small and focused; write the *why* in the commit body.
```

Step 2 changes no file: the tag `v0.3.0` names your commit `chore: release 0.3.0`.